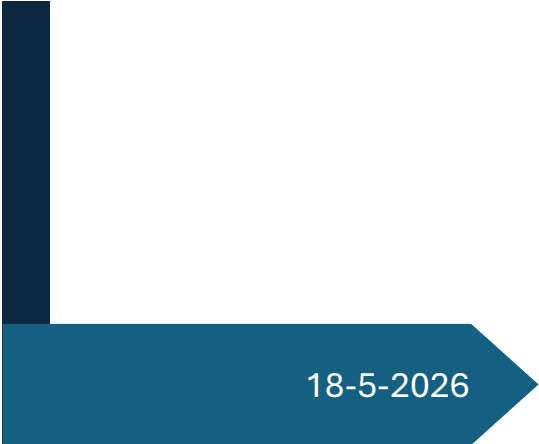
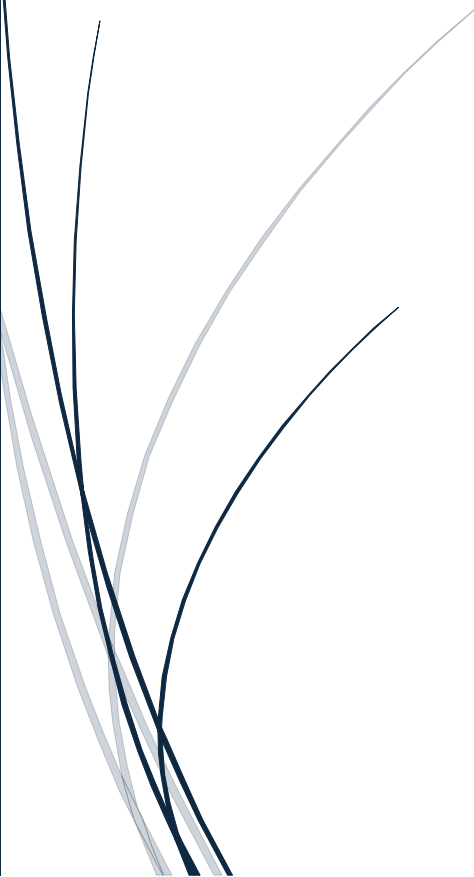


A dark blue vertical bar on the left side of the page. A blue arrow points to the right from the bar, containing the date.

18-5-2026

Documentación

Desarrollo de una API
REST de Gestión de
Productos y Carrito de
Compras con Django
REST Framework

Several thin, curved lines in dark blue and light grey that sweep upwards from the bottom left towards the center of the page.

Nombre del Estudiante: Daniel Altamirano
INSTITUTO SUPERIOR TECNOLÓGICO CEMLAD

Índice

1. Introducción
2. Objetivos
3. Marco Teórico
4. Desarrollo del Proyecto
5. Arquitectura del Sistema
6. Endpoints Implementados
7. Seguridad y Autenticación JWT
8. Documentación Swagger/OpenAPI
9. Pruebas Automatizadas
10. Resultados Obtenidos
11. Conclusiones
12. Recomendaciones
13. Referencias Bibliográficas

1. Introducción

Actualmente las aplicaciones modernas requieren sistemas capaces de intercambiar información de manera rápida, segura y organizada. Las APIs REST se han convertido en uno de los métodos más utilizados para permitir la comunicación entre aplicaciones web, móviles y servicios externos.

En la presente actividad se desarrolló una API REST para la gestión de productos y carrito de compras utilizando Python junto con Django REST Framework. El sistema fue diseñado siguiendo principios REST, permitiendo realizar operaciones de autenticación, administración de productos y manejo de carritos de compra.

El proyecto implementa funcionalidades importantes como autenticación mediante JWT, documentación automática con Swagger/OpenAPI y pruebas automatizadas utilizando pytest. Estas características permiten construir un sistema más seguro, organizado y fácil de mantener.

El desarrollo de esta actividad permitió fortalecer conocimientos relacionados con el ecosistema backend de Python, el uso de APIs REST y las buenas prácticas de desarrollo modernas.

2. Objetivos

2.1 Objetivo General

Desarrollar una API REST funcional para la gestión de productos y carrito de compras utilizando Django REST Framework, aplicando principios REST, autenticación JWT y pruebas automatizadas.

2.2 Objetivos Específicos

- Implementar autenticación segura mediante JWT.
- Desarrollar operaciones CRUD para productos.
- Gestionar carritos de compra asociados a usuarios.
- Documentar automáticamente la API con Swagger/OpenAPI.
- Implementar pruebas automatizadas para validar el funcionamiento del sistema.
- Aplicar buenas prácticas de organización y desarrollo backend.

3. Marco Teórico

3.1 API REST

REST (Representational State Transfer) es un estilo de arquitectura utilizado para diseñar servicios web. Este modelo permite que diferentes sistemas puedan comunicarse mediante solicitudes HTTP utilizando métodos como GET, POST, PUT, PATCH y DELETE.

Las APIs REST se caracterizan por ser ligeras, escalables y fáciles de integrar con aplicaciones frontend, aplicaciones móviles y otros sistemas.

Entre las principales ventajas de las APIs REST se encuentran:

- Separación entre cliente y servidor.
- Facilidad de integración.
- Uso de formatos estándar como JSON.
- Escalabilidad.
- Flexibilidad para múltiples plataformas.

3.2 Django

Django es un framework de desarrollo web creado en Python que permite construir aplicaciones de manera rápida y segura. Este framework incluye herramientas integradas para autenticación, manejo de bases de datos, administración y seguridad.

Una de las principales ventajas de Django es que reduce considerablemente el tiempo de desarrollo gracias a sus componentes reutilizables.

3.3 Django REST Framework

Django REST Framework (DRF) es una extensión de Django diseñada específicamente para construir APIs REST.

DRF facilita la creación de endpoints, serialización de datos, autenticación y permisos. Además, permite estructurar aplicaciones backend de manera organizada y escalable.

Entre sus principales características se encuentran:

- Serialización automática de modelos.
- Soporte para autenticación.
- Manejo de permisos.
- Navegación de APIs.
- Integración con Swagger/OpenAPI.

- Validaciones automáticas.

3.4 JWT (JSON Web Token)

JWT es un mecanismo de autenticación basado en tokens. Cuando el usuario inicia sesión correctamente, el sistema genera un token firmado digitalmente que será utilizado para acceder a rutas protegidas.

La autenticación JWT es ampliamente utilizada porque:

- No requiere mantener sesiones activas en el servidor.
- Mejora el rendimiento.
- Facilita la autenticación en aplicaciones distribuidas.
- Permite mayor escalabilidad.

En este proyecto se utilizó la librería SimpleJWT para implementar autenticación segura.

3.5 Swagger y OpenAPI

Swagger es una herramienta utilizada para documentar APIs REST de manera visual e interactiva.

Gracias a Swagger, los desarrolladores pueden:

- Visualizar todos los endpoints.
- Probar solicitudes directamente desde el navegador.
- Revisar parámetros y respuestas.
- Facilitar la integración con otros sistemas.

OpenAPI es el estándar utilizado para describir APIs REST.

4. Desarrollo del Proyecto

4.1 Descripción General

El sistema desarrollado corresponde a una API REST orientada a la gestión de productos y carritos de compra.

El proyecto fue desarrollado utilizando Python, Django y Django REST Framework. La aplicación permite registrar usuarios, autenticar sesiones mediante JWT y administrar productos y carritos de compra.

El sistema fue estructurado utilizando una arquitectura modular para facilitar el mantenimiento y la escalabilidad.

4.2 Tecnologías Utilizadas

Python	Lenguaje de programación principal
Django	Framework backend
Django REST Framework	Desarrollo de API REST
SimpleJWT	Autenticación JWT
drf-spectacular	Documentación Swagger/OpenAPI
pytest	Pruebas automatizadas
SQLite	Base de datos utilizada durante el desarrollo

4.3 Estructura del Proyecto

El sistema fue organizado en diferentes módulos para separar responsabilidades.

Principales componentes:

- Configuración general del proyecto.
- Aplicación de autenticación.
- Aplicación de productos.
- Aplicación de carrito de compras.
- Configuración de rutas.
- Serializers.
- Views.
- Tests.

Esta estructura permite mantener un código más limpio y ordenado.

4.4 Variables de Entorno

El sistema utiliza variables de entorno mediante archivo `.env` para almacenar información sensible como:

- SECRET_KEY
- DEBUG
- DATABASE_URL

Esto mejora la seguridad del proyecto y evita exponer configuraciones críticas.

4.5 Manejo de Dependencias

Las dependencias del proyecto fueron organizadas mediante un archivo `requirements.txt`, permitiendo instalar fácilmente todas las librerías necesarias.

Entre las dependencias principales se encuentran:

- django
- djangorestframework
- djangorestframework-simplejwt
- drf-spectacular
- pytest-django
- python-decouple

5. Arquitectura del Sistema

5.1 Arquitectura General

La arquitectura del sistema sigue un modelo cliente-servidor.

El cliente realiza solicitudes HTTP hacia la API REST y el servidor procesa dichas solicitudes devolviendo respuestas en formato JSON.

La arquitectura implementada permite separar claramente:

- Lógica de negocio.
- Manejo de datos.
- Autenticación.
- Exposición de endpoints.

5.2 Modelos Principales

Usuario

Representa los usuarios registrados dentro del sistema.

Producto

Almacena la información relacionada con productos disponibles.

Campos principales:

- name
- description
- price
- stock
- created_at

Carrito

Representa el carrito asociado a cada usuario.

CartItem

Representa los productos agregados al carrito junto con sus cantidades.

5.3 Reglas de Negocio Implementadas

El sistema implementa varias validaciones importantes:

- Cada usuario posee un único carrito.
- No se pueden agregar productos sin stock.
- Si el producto ya existe en el carrito se incrementa la cantidad.
- El sistema calcula automáticamente el total del carrito.

Estas reglas ayudan a mantener la integridad de los datos y el correcto funcionamiento del sistema.

6. Endpoints Implementados

6.1 Autenticación

Registro de Usuario

Endpoint:

POST /api/auth/register/

Permite registrar nuevos usuarios utilizando username, email y contraseña.

Obtención de Token JWT

Endpoint:

POST /api/auth/token/

Permite iniciar sesión y obtener:

- Access Token
- Refresh Token

Renovación de Token

Endpoint:

POST /api/auth/token/refresh/

Permite generar un nuevo access token utilizando el refresh token.

6.2 Gestión de Productos

Listar Productos

GET /api/products/

Permite visualizar todos los productos disponibles.

Obtener Producto por ID

GET /api/products/{id}/

Muestra el detalle de un producto específico.

Crear Producto

POST /api/products/

Requiere autenticación.

Actualizar Producto

PUT /api/products/{id}/

Permite modificar completamente un producto.

Actualización Parcial

PATCH /api/products/{id}/

Permite modificar únicamente ciertos campos.

Eliminar Producto

DELETE /api/products/{id}/

Elimina un producto existente.

6.3 Carrito de Compras

Ver Carrito

GET /api/cart/

Permite visualizar el carrito del usuario autenticado.

Agregar Producto al Carrito

POST /api/cart/items/

Agrega productos al carrito.

Actualizar Cantidad

PATCH /api/cart/items/{id}/

Permite modificar cantidades.

Eliminar Producto del Carrito

DELETE /api/cart/items/{id}/

Elimina un producto del carrito.

Vaciar Carrito

DELETE /api/cart/clear/

Elimina todos los productos del carrito.

7. Seguridad y Autenticación JWT

La seguridad del sistema se implementó mediante JWT utilizando la librería SimpleJWT.

Las rutas públicas permiten únicamente lectura de información básica, mientras que las rutas protegidas requieren autenticación.

Cuando un usuario inicia sesión correctamente, el sistema genera tokens JWT que deben enviarse en cada solicitud protegida.

Este mecanismo mejora la seguridad y evita accesos no autorizados.

Además, el sistema implementa validaciones para:

- Acceso a endpoints protegidos.
- Validación de credenciales.
- Restricción de permisos.
- Protección de operaciones críticas.

8. Documentación Swagger/OpenAPI

La API fue documentada automáticamente utilizando drf-spectacular.

La documentación se encuentra disponible mediante:

- /api/docs/
- /api/redoc/

Swagger permite visualizar:

- Endpoints.
- Métodos HTTP.
- Parámetros.
- Respuestas.
- Autenticación.

Esto facilita el uso de la API por parte de otros desarrolladores.

Además, la documentación interactiva permite probar directamente las solicitudes desde el navegador.

9. Pruebas Automatizadas

Las pruebas automatizadas fueron desarrolladas utilizando pytest.

El objetivo principal de las pruebas es verificar que las funcionalidades principales del sistema trabajen correctamente.

9.1 Casos de Prueba Implementados

Registro de Usuario

Se verificó que un usuario pueda registrarse correctamente.

Registro con Correo Duplicado

Se validó que el sistema impida registrar correos repetidos.

Obtención de Token

Se comprobó que los usuarios autenticados puedan obtener tokens JWT válidos.

Listado de Productos

Se verificó que el listado público funcione sin autenticación.

Creación de Productos sin Autenticación

Se comprobó que el sistema retorne error 401.

Creación de Productos con Autenticación

Se validó la creación correcta de productos.

Agregar Productos al Carrito

Se verificó el funcionamiento del carrito de compras.

Cálculo del Total

Se comprobó que el sistema calcule correctamente el total general.

9.2 Cobertura de Código

El proyecto utiliza pytest –cov para generar reportes de cobertura.

La cobertura permite identificar qué partes del sistema han sido verificadas mediante pruebas automatizadas.

10. Resultados Obtenidos

El sistema desarrollado logró implementar correctamente todos los requerimientos funcionales solicitados.

Entre los principales resultados obtenidos se encuentran:

- API REST funcional.
- Autenticación JWT operativa.
- CRUD completo de productos.
- Gestión de carrito de compras.
- Documentación automática con Swagger.
- Validaciones de seguridad.
- Pruebas automatizadas.
- Arquitectura organizada y escalable.

Además, el proyecto permitió comprender de mejor manera el ecosistema backend de Python y las ventajas de Django REST Framework para el desarrollo rápido de APIs.

11. Conclusiones

El desarrollo de esta actividad permitió adquirir conocimientos prácticos sobre la creación de APIs REST utilizando Django REST Framework.

Se logró comprender cómo estructurar aplicaciones backend de manera organizada, implementando autenticación, validaciones y documentación automática.

El uso de JWT permitió desarrollar un sistema de autenticación moderno y seguro, mientras que Swagger facilitó considerablemente la visualización y pruebas de los endpoints.

Las pruebas automatizadas ayudaron a verificar el correcto funcionamiento del sistema y permitieron detectar errores de forma temprana.

En comparación con tecnologías trabajadas anteriormente como Java y Spring Boot, Django REST Framework presentó una curva de desarrollo más rápida y sencilla para proyectos pequeños y medianos.

Finalmente, esta práctica permitió fortalecer habilidades relacionadas con APIs REST, seguridad, organización de proyectos backend y buenas prácticas de desarrollo.

12. Recomendaciones

- Implementar paginación en el listado de productos.
- Agregar roles y permisos más avanzados.
- Incorporar Docker para despliegues.
- Utilizar PostgreSQL en ambientes productivos.
- Implementar pruebas adicionales de rendimiento y seguridad.
- Incorporar integración continua (CI/CD).

13. Referencias Bibliográficas

Django Software Foundation. (2025). *Django documentation*.
<https://docs.djangoproject.com/>

Encode OSS Ltd. (2025). *Django REST framework documentation*.
<https://www.django-rest-framework.org/>

OpenAPI Initiative. (2025). *OpenAPI specification*. <https://www.openapis.org/>

SimpleJWT Contributors. (2025). *Simple JWT documentation*. <https://django-rest-framework-simplejwt.readthedocs.io/>

pytest-dev Team. (2025). *pytest documentation*. <https://docs.pytest.org/>

Melé, A. (2023). *Django 4 by Example*. Packt Publishing.

Fielding, R. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine).

<https://chatgpt.com>

Repositorio del proyecto:

<https://github.com/Daniel-Metal/cemlad-teaching-django.git>